p1062R0
Diet Graphics

Draft Proposal, 7 May 2018

This version:
http://wg21.link/P1062r0

Issue Tracking:
GitHub

Authors:
Bryce Adelstein Lelbach
Olivier Giroux
Zach Laine
Corentin Jabot
Vittorio Romeo

Audience:
WG21

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

The proposed programmatic 2D drawing library is not a good fit for C++.

Table of Contents

§ 1. Acknowledgments

We'd like to acknowledge:

- Michael Spencer and Billy Baker, who provided invaluable feedback at a bar.
- Everyone involved in the discussion of [P0267r7] at the Jacksonville 2018 meeting.

§ 2. Foreword

The authors of [P0267r7], SG13, and everyone else who has worked on the proposed 2D drawing Technical Specification, have put a great amount of work, time and energy in that proposal. We, the authors of this paper, greatly appreciate their contributions to C++. We also greatly value them, personally, as they are simply excellent people.

This paper was difficult to write, because we have no intention to malign their hard work.

We do not have a negative opinion of the particular library in question. We do, however, question whether it should be included in the C++ International Standard.

However, we feel obligated to share our technical perspective on [P0267r7]. In this paper, we explain why we believe that [P0267r7], "A Proposal to Add 2D Graphics Rendering and Display to C++" is not a good fit for C++.

[P0267r7] proposes a programmatic 2D drawing library: it provides programmatic C++ interfaces for drawing **art assets** by manipulating C++ objects and then displaying them on a surface.

§ 3. Utility

Let's take a step back and discuss **why** some on the committee have a desire to standardize a 2D drawing library.

A few different use cases have been espoused during the lifetime of this proposal:

- Improving the teachability of C++.
- Providing programmers with an easy and portable interface suitable for simple 2D graphics tasks.

§ 3.1. Teaching

Some of the motivation behind [P0267r7] is a desire to provide a powerful teaching tool for C++. For new programmers, especially younger programmers, writing command-line applications may not be particularly intuitive or exciting. Being able to see and interact with your code is far more natural. Building graphical applications can help the student connect the code they are writing to the real world. After all, most non-programmers primarily interact with software that has a graphical interface of some sort.

But why does this need to be in the standard? There are, of course, many high-quality 3rd party C++ libraries that provide these features. But, C++ also lacks a standard build system, packing system and a centralized source of packages. This can make it notably more difficult to start using, say, a 3rd party 2D drawing library in C++.

A common criticism of C++ is that the C++'s standard library is not as rich as the core libraries of other languages such as Java, Python, Javascript, Rust, Swift, etc. But is the answer to that to try and standardize everything? Many languages with diverse library ecosystems have them because they have package management; we do not.

We believe it is questionable if the proposed drawing interface actually facilitates teaching. Let's consider an example. Suppose you are teaching a group of teenagers with no programming experience and you want to change that. There are a number of common starting points, but let's use the one that is likely most popular: building a simple game.

Suppose we want our students to start by building a simple animation (later, we'll add keyboard input in to make it a game) where a single character moves around on a screen.

Where do we start?

- First, they need to get a window, and a surface in that window that they can draw in.
- Then, they need to programmatically place the character on the screen at some starting position. The character could be represented as an instance of a class type, whose constructor puts it in the starting position,
- Next, they'll want to move the character around by programmatically updating its position and redrawing the screen. This could be done by adding methods to the character class: `move_left`, `move_right`, `move_up`, `move_down`, etc.

Eventually, we'd want to replace the static animation with movement based on keyboard input. Then we would add other game entities (other characters, obstacles, etc) and game logic for how they would interact.

What we're doing in this curriculum is **teaching programming in C++ using graphics**.

This is not something that the proposed 2D drawing interface enables. [P0267r7] gives you a mechanism to **programmatically draw graphics using C++**. For example, you could programmatically draw the character in the above example.

But, why would you want to have students do this? The drawing of the character is not what we want to focus on; instead we want to build application logic that programmatically manipulates different objects (**assets**) on a surface. Instead of drawing assets programmatically in C++, we could have the students use an image editing tool to create an image (JPG, SVG, etc) to use as their art asset.

Drawing even simple images with a programmatic drawing interface will be verbose, and thus distracting from the core curriculum. If one was to design a curriculum around this proposal, students would have to learn a lot about computer graphics in addition to C++. Our goal is for students to learn C++ programming; graphics should be a teaching aid, not the main focus.

We question whether a drawing library would be a useful teaching aid. A simple facility for building graphical interfaces that supports and leverages established graphics standards and formats would

allow programmers to use common image editing tools to generate art assets instead of having to express them programmatically in C++.

§ 3.2. Enabling Graphics Programming

There are two classes of graphics programmers:

- **Graphical Application Developers (Not Always Experts)** build graphics applications, such as games. They are mainly concerned with developing application logic that manipulates graphics asset but they do not typically programmatically build their assets with lower-level drawing interfaces. The assets they work with are developed in common formats such as Postscript or SVG by either the graphical application developers, artists that they collaborate with, asset banks, etc.
- **Graphical Framework Developers (Experts)** build graphics frameworks, engines, and libraries that graphical application developers use.

Naturally, there are many more graphical application developers than there are framework developers.

§ 3.2.1. Graphical Application Developers

Graphical application developers are application programmers who build (usually interactive) software which displays visual output. Some examples include games, cartography software, and scientific visualization. Most graphical application developers do not regularly produce the art assets they use programmatically. Instead, they consume assets produces upstream by artists, taken from asset banks, etc.

A common concern when graphical application developers decide which APIs to use is what their **asset toolchain** will be - e.g. what tools will generate or what sources will provide the art content. Graphical application programmers are going to be inclined to use APIs that don't require explicit conversion, or worse - require them to write the conversion code themselves.

Programmers developing graphical applications have many excellent options in 3rd party libraries, many of which have far greater capabilities than [P0267r7]. What would attract them to [P0267r7] over the established alternatives?

3.2.2. Graphical Framework Developers

Now let's talk about the expert graphics programmers who are building graphics libraries and frameworks, which graphical application developers use. Those graphical application developers want to pass the assets they created in existing established formats, such as Postscript or SVG. For the frameworks to use the current proposed API, they would have to convert those assets to the [P0267r7] path representation. This would be unnecessary and inefficient, because the lower level APIs that [P0267r7] would be built on top of probably support those formats natively. Additionally, since the proposed API is not as extensive as SVG or Postscript, some things will not be easily expressible with [P0267r7]. Suppose we create an asset in SVG using a type of arc that SVG natively supports but [P0267r7] does not. You would have to figure out how to map that primitive to the [P0267r7] primitives.

Some have suggested that professional graphics programmers would use the proposed 2D drawing library. Based on the author's collective knowledge of the graphics community, we strongly believe this statement is not true. Game/computer vision/visualization/etc developers are not going to use the proposed library. **Graphics has its own standards, and graphics programmers don't want us to make another one.**

4. Design Issues

This section describes a number of high-level, broad technical concerns with [P0267r7]. This list is not comprehensive.

4.1. Lack of Batching Operations

The current design of the library severely limits implementation options. Essentially, due to the lack of a true batching API, the current design cannot efficiently utilize GPUs.

GPUs are, fundamentally, bandwidth-optimized processors, while CPUs are latency-optimized processors. Additionally, CPU to GPU communication typically has much greater latency than communication between threads and processes that reside on a single CPU (or even on between multiple CPUs within a single system).

The key to efficient utilization of a GPU is to minimize the frequency of communication between the CPU and GPU and to maximize the size of the task given to the GPU by the CPU in each communication.

This pushes GPU programming towards bulk/batched interfaces. The latency cost of dispatching a single work item is high, so dispatching multiple work items in a single dispatch is desirable as it amortizes the cost.

Modern 2D graphics libraries typically provide fully batched interfaces, which allow a programmer to describe an entire scene (paths, strokes, fills, etc) in a data structure and then hand it off in a single dispatch.

The current design of the library does not follow this trend. There is a mechanism for buffering paths, `path_builder`, but this is not sufficient as there is no mechanism for buffering strokes, fills, etc.

For example, if I wanted to draw two figures, each with a different brush, I have no way of expressing this as single buffered rendering command:

```
auto sfc = make_image_surface(format::rgb32, 1024, 1024);

path_builder pb0{};

pb0.new_figure({ 20.0f, 20.0f });
pb0.line({ 100.0f, 20.0f });
pb0.line({ 20.0f, 100.0f });
pb0.close_figure();

// Render
sfc.stroke(brush_a, pb, nullopt, stroke_props{ 10.0f }, nullopt, aliased);

path_builder pb1{};

pb1.new_figure({ 20.0f, 20.0f });
pb0.line({ 20.0f, 100.0f });
pb0.line({ 100.0f, 20.0f });
pb1.close_figure();

// Render
sfc.stroke(brush_b, pb, nullopt, stroke_props{ 10.0f }, nullopt, aliased);
```

P0267r7's current model will largely prevent an efficient GPU implementation.

§ 4.2. Text and Unicode

In [N3791], the paper that began work on the proposed drawing library, it was stated that the library should be able to draw text.

The current proposal, [P0267r7], has a section on text rendering and display. The section in it's entirety is reproduced below:

> [*Note:* Text rendering and matters related to it, such as font support, will be added at a later date. This section is a placeholder. The integration of text rendering is expected to result in the addition of member functions to the surface class and changes to other parts of the text. — end note]

Why is this feature missing from the library? This component of the library would depend on the standardization of a text library with Unicode support. There is work ongoing in this space, but it has not yet reached maturity.

How much value and utility is there in producing a drawing library that cannot draw text?

§ 4.3. Geometry and Linear Algebra Types

[P0267r7] proposes a number of geometry and linear algebra primitives, such as:

- `basic_point_2d`, a 2D point type whose element type is `float`.
- `basic_matrix_2d`, which represents three by three matrices, whose element type is `float`.
- `basic_bounding_box`, which represents a rectangular space, whose element type is `float`.
- `basic_circle`, which represents a circle, whose element type is `float`.

These abstractions are far more general than 2D drawing. Why are they being designed in this paper, and why are they being designed specifically for 2D drawing? Why do they have a specific element type instead of being parameterized on element type? Adopting these types is likely to lead to inconsistencies and pain down the road.

`basic_point_2d` is specified to have a specific number of dimensions (2) and a specific element type (`float`). Even within the field of 2D graphics, this seems unnecessarily specific. Other proposals in flight have introduced general purpose integer point types with arbitrary dimensionality ([P0009r5]).

Likewise, `basic_matrix_2d` is not a generic linear algebra type, but instead a 2D matrix of a prescribed size, layout and element type. Instead of having an indexing operator or method that takes

an index as a parameter, this type an individual method for accessing each element (`m00`, `m01`, etc). Without a `operator[0][1]` or `get(0, 1)`, indexing with a runtime value is difficult. If `basic_matrix_2d` was considered in a vacuum, we do not believe it would pass muster for standardization. This very specific type is likely not generic or robust enough for even the domain of 2D vector graphics, much less the wider community.

`basic_bounding_box` and `basic_circle` likewise suffer from a lack of genericity.

This is not generally how we design library facilities. Other domains that care about multi-dimensional coordinate types and linear algebra would be better served if we spent time designing generic, general purpose facilities in an effort independent of 2D drawing, but aware of its requirements.

Before we spend additional time standardizing these dependencies of [P0267r7], perhaps we should consider whether that time would be better spent standardizing general purpose facilities that a 2D drawing library could be built on top of.

§ 4.4. New Container Type

[P0267r7] proposes a new container type, `basic_path_builder` (roughly equivalent to a `std::vector<typename basic_figure_items<GraphicsSurface>::figure_item>`) which seem unnecessary. The interface of this type is largely additive on top of `vector`.

The need and justification for a new container type is unclear, as is the need to constrain users to storing sequences of figure items in a particular container type. Generic algorithms that operate on iterators (or ranges) of figure items would be a far more flexible design.

§ 5. Priorities

§ 5.1. Committee Resources are Limited

An unfortunate reality of our work is that committee resources and time are finite. Thus, we must prioritize and even discard proposals either explicitly, or implicitly by never giving them time. Directions for ISO C++ attempts to describes how that sorting and prioritizing should happen.

We must consider whether spending additional time on this proposal is the right investment for us. In its current form, the proposal, which is limited to drawing shapes, is about 150 pages long. It’s one of the largest proposals currently in flight. [N3791], the paper which initiated work on a graphics proposal, stated that this feature should "ship in two years". **That was in 2013.**

We should be mindful of the sunk cost fallacy. The proposal is still in LEWG and it seems clear that it needs additional time there before it advances, as there are many existing technical issues, some of which are detailed above. Both at the Jacksonville 2018 meeting during the LEWG discussion of this proposal and during subsequent discussions, committee members with experience sitting in LWG estimated that it would take the equivalent of an entire meeting of LWG's time to get the proposal out for a Technical Specification. Assuming that estimate is roughly accurate, it indicates a substantial resource investment. We only have nine meetings in between each International Standard. Approximately two to three of those meetings get spent on ballot resolution, leaving about six meetings for feature development. **Are we comfortable spending 1/6th of the time that LWG would spend on an International Standard development cycle to get this feature into a Technical Specification?**

There are many important library facilities that are currently missing from C++ which have greater and broader utility, such as ranges, networking, executors, text, unicode. A number of those have been discussed in this proposal, as a drawing library depends on having these features.

The committee is increasingly backlogged these days. The reality, as pointed out in Directions for ISO C++, is that we must evaluate and prioritize how we spend our time. It is not "free" for us to continue investing time on [P0267r7]. In fact, quite the opposite; it comes with a great cost, even if the end goal is just a Technical Specification.

Given that:

- there are no clear users or demand for this feature from the community,
- there is higher impact work that a 2D drawing library depends on,
- there is higher impact work in general, and
- the committee does not have infinite time,

we believe that we should not continue pursuing [P0267r7].

§ 5.2. What Belongs in The Standard?

The C++ Standard Library is not a library. The C++ Standard is, well, a standard; a normative document that describes in the most detailed and unambiguous way possible a universal and portable programming language implemented by many different vendors on a vast and varied set of platforms and used by ~5 million programmers. The C++ Standard Library is not a library; it is a specification for a library.

The bar for entry should be high. The things that go into the C++ Standard should be cross-cutting across multiple domains within our community. We should seek to standardize best practices and the things that would be terribly inconvenient to live anywhere but the standard. The features we put into the C++ Standard should be be implementable across all modern platforms by all the vendors within our ecosystem, within reason.

What should, or should not be included in the C++ standard library is a very important question.

- Containers?
- Filesystem Access?
- Networking?
- Unicode?
- JSON?
- XML?
- Database Access?
- Serialization?
- 2D Graphics?
- 3D Graphics?
- Crypto?

At some point, we need to draw a line. What things are fundamental and absolutely need to work everywhere? Those are the things that need to go in.

Some have suggested that the proposed 2D drawing Technical Specification could remain as just that - a Technical Specification. Technical Specifications do not necessarily need to eventually be merged into the International Standard. But, we have to be aware of the risk of segmenting the language (which would be harmful to the community) by creating a multitude of Technical Specifications that are not intended for unification with the International Standard. Technical Specifications are not a substitute for a package management system that has a principled approach to managing prerequisites and dependencies.

Speaking of package management... There many high quality C++ 2D graphics libraries out there. Why aren't they good enough?

Perhaps 2D graphics isn't the problem, but a symptom of a larger problem. We are all aware - or should be - that one of the major pain points of C++ programming today is package management. This was one of the leading responses from the C++ developer survey.

If the C++ community had a first class solution for package management - something with widespread adoption by users and implementers - would we be thinking about putting a 2D drawing library into the C++ Standard?

As the C++ committee, package management may well be outside of our scope and mandate. As leaders of the C++ community, it certainly is not.

§ 6. Suggestions

- **The C++ committee should not pursue a programmatic 2D drawing library at this time.**
- **Text, Unicode, geometry and linear algebra libraries are prerequisites of a 2D drawing library that seem reasonable to consider for standardization.**
- **We, the leaders of the C++ community, need to get serious about package management (not necessarily within the confines of standard).**

§ 7. Alternative Minimal Graphics

If the committee feels strongly that we want prioritize a **minimal** 2D vector graphics library that:

- Gives you a portable way to get a window
- Loads assets in an established vector graphics format
- Draws them on a surface in the window

We suggest the following minimal design and thin interface:

```
namespace std::graphics {

  struct display_exception;

  struct asset {
    explicit asset(string const& xml);
    explicit asset(filesystem::path const& file);
    ~asset();
  };

  template<class Function>
  static void process_events(double timeout, Function f);

  template<class T>
  struct point { T x, y; };

  template<class T>
  struct rectangle { point<T> top_left, bottom_right; };

  struct surface {
    void clear();
    void apply(asset const& a, rectangle<float> /*window subrect*/);
  };

  struct window {
    window(rectangle<int> rect = {}, string const& title = "");
    ~window();

    void on_key_down(function<void(int /*key code*/)> action);
    void on_key_up(function<void(int /*key code*/)> action);
    void on_pointer_pos(function<void(float /*normalized x*/, float /*norma
    void on_pointer_down(function<void(int /*button code*/)> action);
    void on_pointer_up(function<void(int /*button code*/)> action);

    void on_draw(function<void(surface&/*render target*/)> action);
    void on_close(function<void()> action);
    void on_place(function<void(rectangle<int>/*display subrect*/)> action)

    void place(rectangle<int> rect);
    void redraw();
    bool closed() const;

    using native_handle_type = implementation-defined;
```

```
    native_handle_type native_handle();
  };

}
```

Objects of class `asset` encapsulate a vector or raster asset: a W3C SVG graph, a PNG image, etc. Constructors are provided that consume strings (W3C SVG graph format, etc) and files (textual representation of a graph in the W3C SVG XML format, binary PNG content, etc). Much like objects of class `regex`, these can be parsed once and used often, to amortize the cost of constructing the graph. [Note: Objects of class `asset` are akin to figures. – End note] [Note: Implementations may define additional asset formats. – End note]

The free function `process_events` shall be invoked in the main thread, and repeatedly executes the following steps: 1.) Blocks waiting for the next window event, or until `timeout` seconds have elapsed. 2.) Processes all available window events. 3.) Invokes `f`, it shall be `Callable` with no arguments and return `bool`. 4.) If `f` returned false, `process_events` returns.

Objects of class `window` encapsulate implementation-defined display windows. A display window is user-visible rectangular grid of pixels that can be the target of user interaction events, such typing, pointing, moving, resizing and closing.

The constructor `window::window` creates a visible display window with suggested screen footprint `rect` in pixels, and suggested name title. Implementations should ensure that these parameters are respected, if possible. The member functions have the following semantics:

- The member functions `window::on_*` set the event handler for different event. Each takes a `Callable` operand that replaces the previous handler, if any. [*Note:* The only way to draw on a display window is by registering a handler with `on_draw`, in order to get access to a surface object. – *end note*]
- The member function `window::place` raises a place event on the window, with suggested screen footprint `rect`.
- The member function `window::redraw` raises a draw event on the window. [*Note:* This causes the handler registered with `on_draw` to eventually execute. – *end note*]
- The member function `window::closed` returns true if the display window has been closed.
- The member function `window::native_handle` returns an object of type `native_handle_type` suitable for use with implementation-defined capabilities, such as advanced rendering capabilities. Implementations may require that no operation to the display

window be performed through the `native_handle` once any operation has been performed through a `surface` object.

Objects of class `surface` encapsulate the rendering capabilities of a display window. A display window may only render objects of class `asset`, and only as an effect of executing a function registered with on_draw. A surface presents only two operations:

- The member function `surface::clear` invalidates the contents of the display window. The meaning of this operation is implementation-defined.
- The member function `surface::apply` renders an object of class asset with relative footprint `rect`.

An example:

```
#include <graphics>

template<class T>
using rectangle = std::graphics::rectangle<T>;
using asset = std::graphics::asset;
using window = std::graphics::window;
using surface = std::graphics::surface;

int main() {

  window w(rectangle<int>{{0,0},{600,600}});

  asset const a{ my_asset_string_here };
  w.on_draw([&](surface& s) {
    s.clear();
    s.apply(a, rectangle<float>{{0.f,0.f},{1.f,1.f}});
  });
  window::process_events(1./60, [&](){
    if (w.closed())
      return false;
    w.redraw();
    return true;
  });
}
```

§ References

§ Informative References

[N3791]

Beman Dawes. Lightweight Drawing Library - Objectives, Requirements, Strategies. 11 October 2013. URL: https://wg21.link/n3791

[P0009r5]

H. Carter Edwards, Daniel Sunderland, David Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos. Polymorphic Multidimensional Array Reference. URL: https://wg21.link/p0009r5

[P0267r7]

Michael B. McLaughlin, Herb Sutter, Jason Zink, Guy Davidson. A Proposal to Add 2D Graphics Rendering and Display to C++. URL: https://wg21.link/p0267r7